

阶段性报告 2

核心目标

自动化的生成更多的“考点”。

- 数据从哪来？
- 如何把收集到的数据变成考点？

类比 —— 自然语言处理（NLP）中的文本生成机制：

词表	静态的 skill_bank.py 中所有的考点类。
上下文	当前已拼装好的前半段题目描述与已有图谱连线。
Next Token Prediction	图引擎通过对比当前节点的输出端口（Provides）与词表中其他节点的输入端口（Requires），进行强类型匹配，拼接出下一步的考点。

阶段瓶颈：随着业务复杂度的增加，如果每次引入新考点都需要人类手工编写带有复杂 Pandas 逻辑的 Python 类，系统将难以实现自动化和规模化扩展。

底层架构重构

为了让大模型能够安全地介入考点生成，我对底座进行了彻底解耦：

- “骨架” (Python代码)：提炼出纯粹的底层数学与数据变换动作。大模型对此层仅有只读权限。
- “灵魂” (JSON 配置)：将具体的业务外衣 (Prompt、判分点、执行参数) 全部抽离进 skills_config.json。大模型可以通过操作 JSON，为通用算子赋予不同的业务含义（例如将一个“乘法算子”配置为“扣税”或“补贴”）。

大模型专属 API 工具链

为了防止大模型直接修改 JSON 导致格式崩溃，我们为其开发了一套专属的“接口沙盒”，强制大模型通过工具调用（Function Calling）来与系统交互：

- 查询能力：search_skills（查重、检索）、list_available_operators（查看底层数据操作）。
- 图谱操作：create_skill（新考点入库）、connect_skills（考点之间连线）。
- 人类协同 (Human-in-the-loop)：request_new_operator（当现有算子无法满足新逻辑时，向人类提交申请）。

规范全局标准

在真正让流水线运转前，我们解决了两个真实的图谱评测痛点：

- 端口字典 (Port Dictionary)：建立统一的 ports_dict.json 语义规范。大模型在推演连线前必须查阅（如 Financial:PreTax），彻底杜绝了因大模型随意造词导致的“连线失败”问题。

- 隐式陷阱：考点分为“明线”（业务步骤）和“暗线”（脏数据/陷阱）。在正式把“自然语言”提炼为考点之前，先尝试显式地加入陷阱考点。这样保证后面提炼出来的考点包含陷阱。

自动化出题流水线

- 起点：变异孵化。
 - 基于 Masked Language Modeling 思想，截断已有的基础图谱路径（如 $A \rightarrow B$ ），利用大模型的高发散性，结合不同的行业背景续写出全新的自然语言任务分支。
- Step 0: 陷阱显形。
 - 输入：变异产生的全新自然语言任务。
 - 机制：查询可用陷阱库，挑选合适的异常（如“缺失值”），将其润色进原始文本，并在末尾打上 [内部出题指令] 的暗号。
- Step 1: 业务提炼
 - 剔除自然语言中的无用排版或政策废话，将长文本严格切分为原子操作动作（强制声明 Base 根节点，并解析提取暗号转化为 Trap 动作），输出纯净的 JSON 动作序列。
- Step 2: 端口推演
 - 对照全局端口字典，为上一步的每一个动作切片插上强类型的 Requires 和 Provides 插头。如果遇到前所未见的业务维度，则通过 API 规范地注册新端口。
- Step 3 & 4: 匹配与连线入库
 - 基于动作特征和端口，在系统中匹配底层算子。如果没有重复，则调用 create_skill 在 JSON 中落盘，并调用 connect_skills 融入庞大的全局有向无环图，完成图谱的自我扩充。